

Computer Architecture

Lecture 16

Exceptions and Interrupts

- “Unexpected” events requiring change in flow of control
 - Different ISAs use the terms differently
- Exception
 - Arises within the CPU
 - e.g., undefined opcode, syscall, ...
- Interrupt
 - From an external I/O controller
- Dealing with them without sacrificing performance is hard

Handling Exceptions

- Save PC of offending (or interrupted) instruction
 - In RISC-V: Supervisor Exception Program Counter (SEPC)
- Save indication of the problem
 - In RISC-V: Supervisor Exception Cause Register (SCAUSE)
 - 64 bits, but most bits unused
 - Exception code field: 2 for undefined opcode, 12 for hardware malfunction, ...
- Jump to handler
 - Assume at 0000 0000 1C09 0000_{hex}

An Alternate Mechanism

- Vectored Interrupts
 - Handler address determined by the cause
- Exception vector address to be added to a vector table base register:
 - Undefined opcode 00 0100 0000_{two}
 - Hardware malfunction: 01 1000 0000_{two}
 - ...: ...
- Instructions either
 - Deal with the interrupt, or
 - Jump to real handler

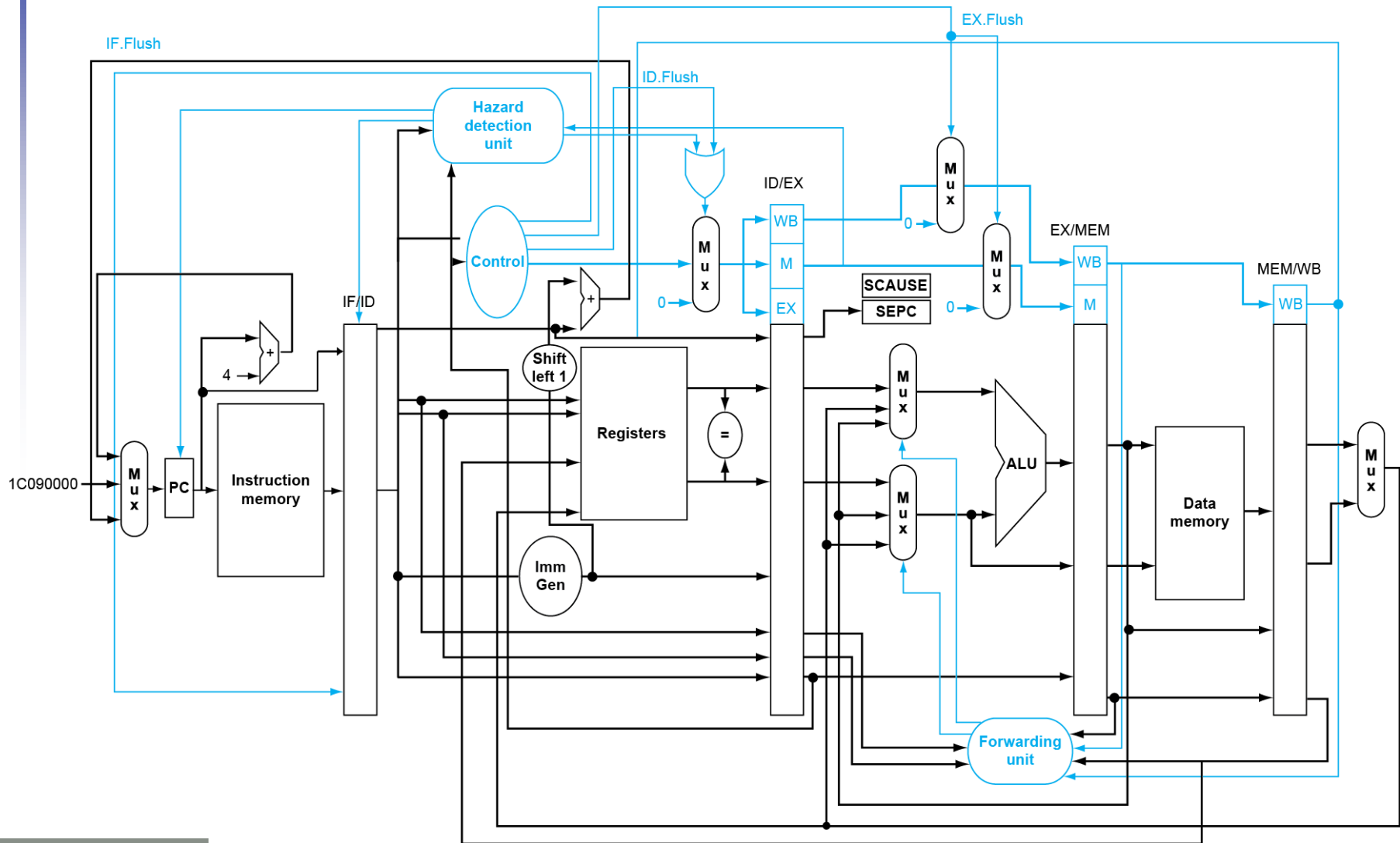
Handler Actions

- Read cause, and transfer to relevant handler
- Determine action required
- If restartable
 - Take corrective action
 - use SEPC to return to program
- Otherwise
 - Terminate program
 - Report error using SEPC, SCAUSE, ...

Exceptions in a Pipeline

- Another form of control hazard
- Consider malfunction on add in EX stage
add x1, x2, x1
 - Prevent x1 from being clobbered
 - Complete previous instructions
 - Flush add and subsequent instructions
 - Set SEPC and SCAUSE register values
 - Transfer control to handler
- Similar to mispredicted branch
 - Use much of the same hardware

Pipeline with Exceptions



Exception Properties

- Restartable exceptions
 - Pipeline can flush the instruction
 - Handler executes, then returns to the instruction
 - Refetched and executed from scratch
- PC saved in SEPC register
 - Identifies causing instruction

Exception Example

- Exception on `add` in

```
40      sub    x11, x2, x4
44      and    x12, x2, x5
48      orr    x13, x2, x6
4c      add    x1,  x2, x1
50      sub    x15, x6, x7
54      ld     x16, 100(x7)
```

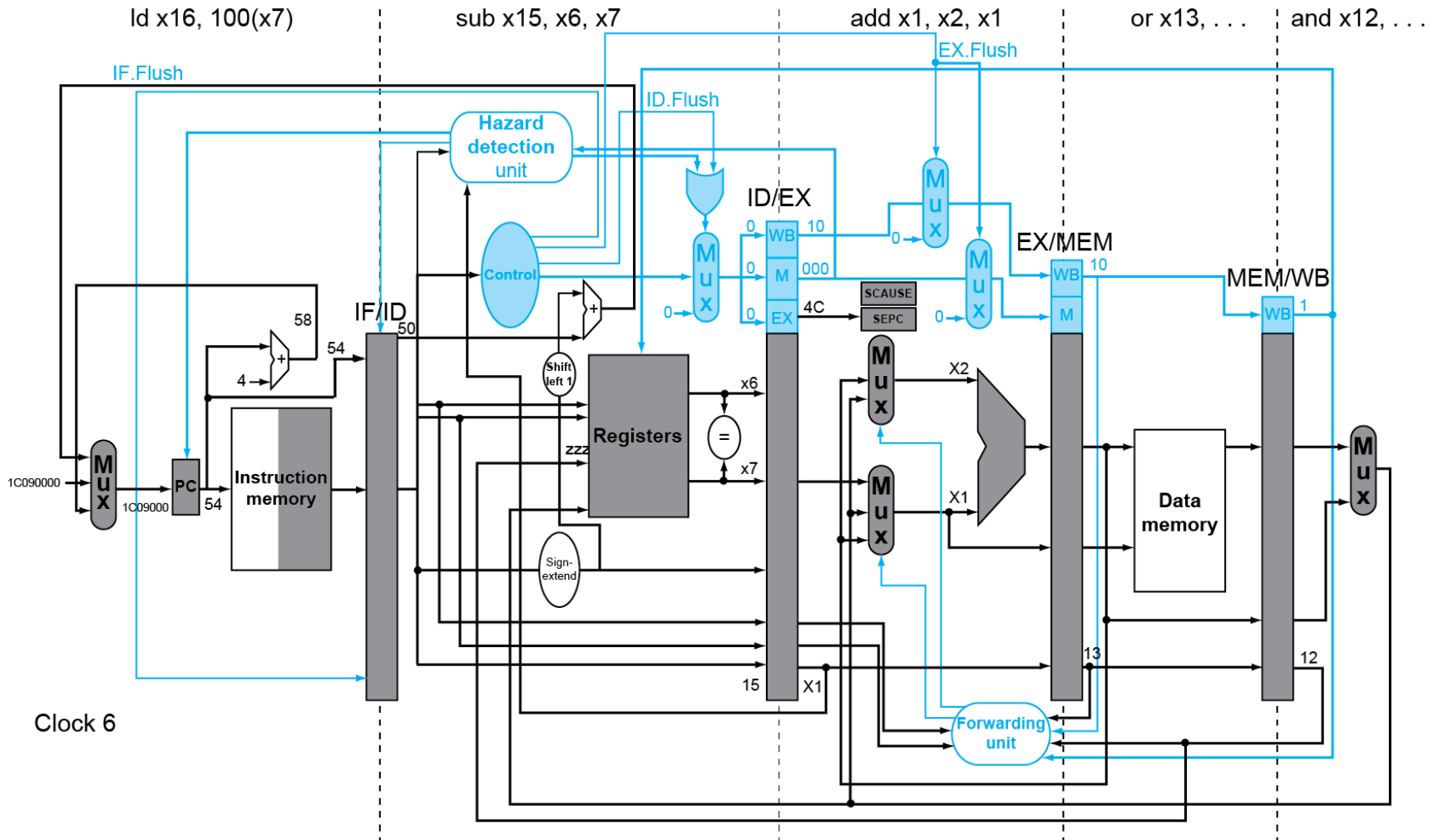
...

- Handler

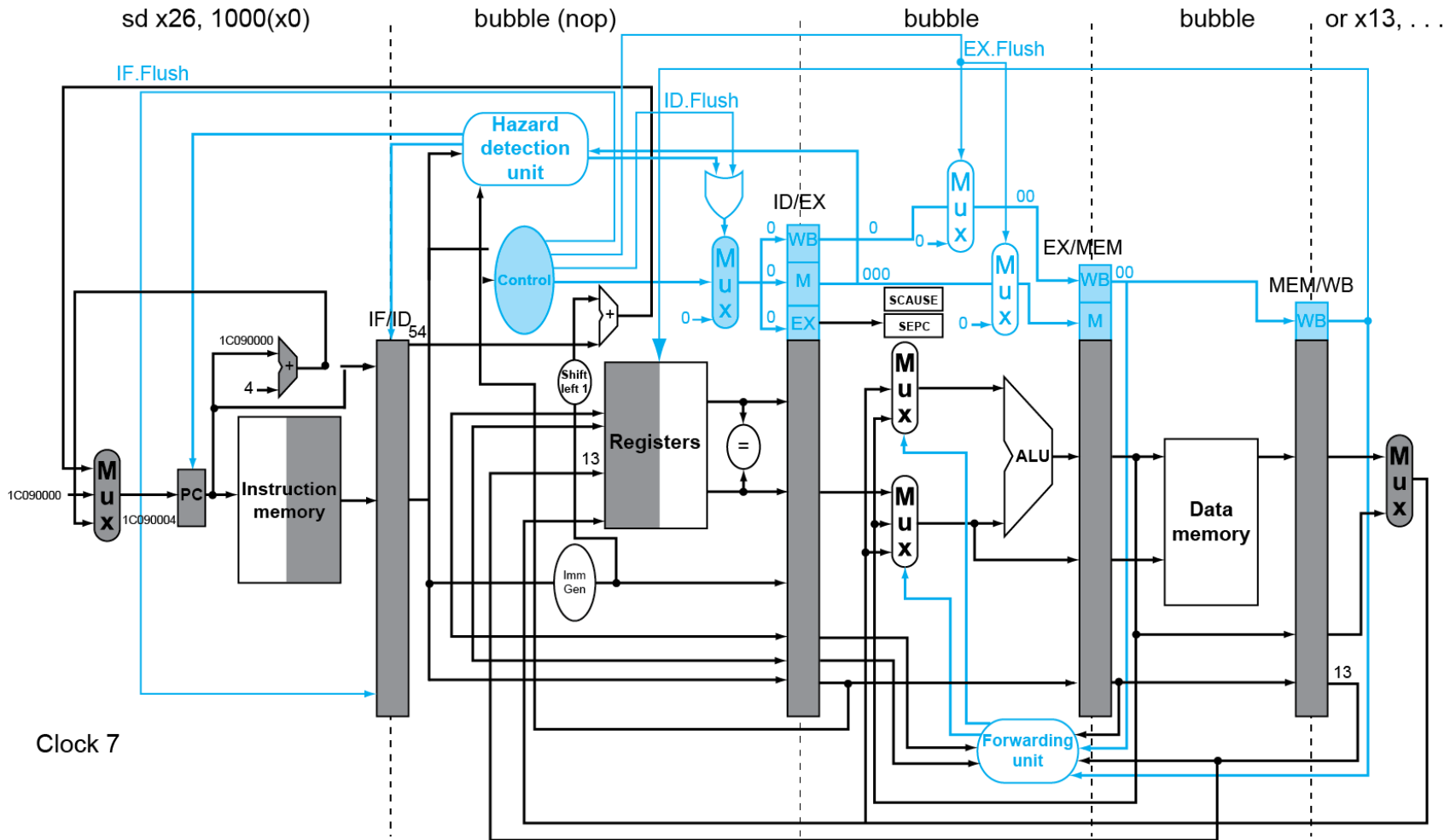
```
1c090000      sd    x26, 1000(x10)
1c090004      sd    x27, 1008(x10)
```

...

Exception Example



Exception Example



Multiple Exceptions

- Pipelining overlaps multiple instructions
 - Could have multiple exceptions at once
- Simple approach: deal with exception from earliest instruction
 - Flush subsequent instructions
 - “Precise” exceptions
- In complex pipelines
 - Multiple instructions issued per cycle
 - Out-of-order completion
 - Maintaining precise exceptions is difficult!

Imprecise Exceptions

- Just stop pipeline and save state
 - Including exception cause(s)
- Let the handler work out
 - Which instruction(s) had exceptions
 - Which to complete or flush
 - May require “manual” completion
- Simplifies hardware, but more complex handler software
- Not feasible for complex multiple-issue out-of-order pipelines